

Prompting Techniques

Proven patterns for getting better answers out of any LLM — from zero-shot and few-shot examples to chain-of-thought reasoning, self-consistency, ReAct, tree of thoughts, reflection, and prompt chaining.

DIFFICULTY LEVEL

Beginner-Intermediate

ESTIMATED READING TIME

60-75 minutes

ESTIMATED PRACTICE TIME

100-130 minutes

PREREQUISITES

Modules 01-02

LEARNING OUTCOMES

By the end of this module, you will be able to: choose between zero-shot, one-shot, and few-shot prompting for a task; apply step-by-step and chain-of-thought prompting to improve reasoning accuracy; use self-consistency to catch unreliable answers; understand the ReAct and Tree of Thoughts patterns conceptually; apply reflection to self-correct output; and chain prompts together for multi-stage tasks.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Zero-shot Prompting
3.2	One-shot Prompting
3.3	Few-shot Prompting
3.4	Step-by-step Prompting
3.5	Chain-of-Thought (Concept)
3.6	Self-Consistency
3.7	ReAct
3.8	Tree of Thoughts (Concept)
3.9	Reflection
3.10	Prompt Chaining
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

Module 02 gave you the raw building blocks of a prompt. This module gives you the *patterns* — proven ways of arranging those building blocks that reliably improve accuracy, consistency, and reasoning quality, especially on harder tasks.

Why is it important?

The same model, given the same underlying task, can go from "wrong" to "right" purely by changing the prompting technique — no fine-tuning, no new model, just a smarter pattern. These techniques are the difference between a hobbyist prompting casually and an engineer prompting deliberately.



Why should AI Engineers learn it?

Different tasks call for different techniques — a simple classification task might only need few-shot examples, while a multi-step math or logic problem benefits enormously from chain-of-thought or self-consistency. Knowing the toolbox means picking the cheapest technique that actually solves the problem, instead of over- or under-engineering every prompt.

Where is it used in the real world?

Coding assistants use step-by-step and ReAct-style patterns to plan before acting. Research and agent tools use tree-of-thoughts-style exploration for hard, open-ended problems. Classification and extraction pipelines lean on few-shot examples for consistency.

Why is it relevant today?

As reasoning models (Module 01, 3.12) and agents (Module 08) become standard, understanding these techniques conceptually — even the ones now partly automated by the model itself — helps you debug and design AI systems that use them under the hood.

Note

Some techniques here (like Chain-of-Thought and Tree of Thoughts) are covered at a conceptual level, since deep implementation details belong to research-level material. The goal is: know what each technique is, when to reach for it, and roughly how it works.

2. Before You Begin

Make sure these are comfortable before continuing:

RECAP

Prompt structure (Module 02) — instructions, context, constraints, and delimiters are the pieces every technique below rearranges and builds on.

RECAP

How LLMs generate text (Module 01, 3.3) — token-by-token generation explains why "thinking out loud" in a prompt (chain-of-thought) actually helps: it gives the model intermediate tokens to condition on before committing to a final answer.



Tip

Try each technique in this module on the same test question — it's the fastest way to feel the real difference each one makes.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

03.1 Zero-shot Prompting

DEFINITION

Zero-shot prompting means asking the model to perform a task with no examples given — just an instruction.

SIMPLE EXPLANATION

You rely entirely on the model's pretrained knowledge and its ability to understand instructions directly, without showing it what a "correct" answer looks like.

💡 Like asking a new hire to "write a professional apology email" with no sample email attached — they draw entirely on general knowledge of what "professional" and "apology" mean.

EVERYDAY EXAMPLE

"Classify this review as positive, negative, or neutral: 'The food was cold but the service was excellent.'" — no examples, just the instruction and the text.

WHY IT MATTERS

Zero-shot is the fastest, cheapest technique — fewer tokens, less setup. It's the right default for simple, well-understood tasks.

COMMON MISTAKES

Using zero-shot for tasks with a very specific or unusual desired format/style, where the model has no way to guess your exact preference.

BEST PRACTICES

Start with zero-shot for simple tasks; upgrade to few-shot only if outputs are inconsistent or don't match your desired style.

SUMMARY

Zero-shot is instruction-only prompting — cheap and often sufficient, but with no guardrails on exact style or format via examples.

13.2 One-shot Prompting

DEFINITION

One-shot prompting means providing exactly one example of the task alongside the instruction, to show the model the desired pattern.

SIMPLE EXPLANATION

A single example anchors format, tone, and structure far more precisely than an instruction alone, without the token cost of multiple examples.

💡 Like showing the new hire one sample apology email as a template — now they have a concrete pattern to follow, not just a description.

EVERYDAY EXAMPLE

"Translate to formal Japanese. Example: 'Thanks!' → 'ありがとうございます。' Now translate: 'See you later!'"

WHY IT MATTERS

One-shot is a lightweight middle ground — better format control than zero-shot, cheaper than few-shot.

COMMON MISTAKES

Choosing an example that's unrepresentative of the range of real inputs, causing the model to overfit its output style to that one narrow case.

BEST PRACTICES

Pick an example that's representative and demonstrates the exact format you want, including edge-case handling if relevant.

SUMMARY

One-shot prompting anchors the desired pattern with a single concrete example — a cheap step up from zero-shot.

📌 3.3 Few-shot Prompting

DEFINITION

Few-shot prompting provides multiple examples (typically 2-10) of the task before asking the model to perform it on new input.

SIMPLE EXPLANATION

More examples let the model infer the underlying pattern more reliably — especially useful when the pattern isn't obvious from a single example, or when you need to demonstrate variety (different formats, edge cases, tones).

```
Review: "Amazing quality, fast shipping!"      → Positive
Review: "Broke after one day, terrible."       → Negative
Review: "It's fine, does what it says."        → Neutral
Review: "Cold food but the staff was great."   → ?
```

💡 *Like showing several graded essays before asking a grader-in-training to grade a new one — a handful of examples teaches the grading pattern far more reliably than just one.*

EVERYDAY EXAMPLE

Sentiment classification, data extraction into a specific format, and style-matching tasks all benefit heavily from 3-5 well-chosen examples.

WHY IT MATTERS

Few-shot prompting is one of the most reliable ways to improve consistency on tasks with a defined, learnable pattern — often without needing any model fine-tuning at all.

COMMON MISTAKES

Using too many examples and blowing through context window/cost unnecessarily. Using examples that are all near-identical, giving no coverage of edge cases.

BEST PRACTICES

Choose diverse, representative examples. Keep the count as low as reliably works — more examples cost more tokens per request.

SUMMARY

Few-shot prompting teaches a pattern through multiple examples — the go-to technique for classification, extraction, and style-matching tasks.

📌 3.4 Step-by-step Prompting

DEFINITION

Step-by-step prompting explicitly instructs the model to break a task into sequential steps before producing the final answer.

SIMPLE EXPLANATION

Instead of demanding the final answer immediately, you ask the model to work through the problem in stages — this gives it more "thinking tokens" to reason through before committing.

💡 *Like asking someone to show their work on a math test instead of just writing the final number — the process itself catches errors that jumping straight to an answer would miss.*

EVERYDAY EXAMPLE

"First, list the key facts. Second, identify what's being asked. Third, work out the answer based on the facts." vs. just "What's the answer?"

WHY IT MATTERS

This is the simplest, most accessible way to improve accuracy on multi-part or reasoning-heavy tasks — no special phrase magic required, just explicit staging.

COMMON MISTAKES

Asking for steps but not actually using them — if you only read the final answer and ignore the reasoning, you lose the chance to catch a flawed step.

BEST PRACTICES

Explicitly number the stages you want the model to follow, especially for tasks with real sequential logic (research, planning, multi-part analysis).

SUMMARY

Step-by-step prompting breaks a task into stages, giving the model room to reason incrementally instead of jumping straight to a possibly-wrong answer.

🔗 3.5 Chain-of-Thought (Concept)

DEFINITION

Chain-of-thought (CoT) prompting asks the model to generate its intermediate reasoning ("thinking out loud") before the final answer, typically triggered by a phrase like "Let's think step by step."

SIMPLE EXPLANATION

CoT is closely related to step-by-step prompting, but specifically refers to the research-established technique of eliciting a visible reasoning trace, which measurably improves accuracy on math, logic, and multi-step problems.

💡 *Like a detective narrating their deductions out loud instead of silently announcing a suspect — narrating the reasoning both improves the reasoning itself and lets you audit it.*

EVERYDAY EXAMPLE

"A store had 23 apples, sold 8, then received 15 more. How many apples now? Let's think step by step." — the model works through $23 - 8 = 15$, then $15 + 15 = 30$, rather than guessing a final number directly.

WHY IT MATTERS

CoT is one of the most well-documented accuracy boosts in prompting research, especially for arithmetic, logic, and multi-hop reasoning tasks.

COMMON MISTAKES

Using CoT for trivial tasks where it just adds cost and latency with no accuracy benefit. Not realizing modern reasoning models (Module 01, 3.12) often do a form of this internally, automatically.

BEST PRACTICES

Reserve CoT for genuinely non-trivial reasoning tasks. If using a reasoning-mode model, explicit CoT prompting is often less necessary — check provider guidance.

SUMMARY

Chain-of-thought elicits visible intermediate reasoning, meaningfully improving accuracy on hard, multi-step problems.

🔁 3.6 Self-Consistency

DEFINITION

Self-consistency runs the same chain-of-thought prompt multiple times (often at non-zero temperature) and takes the most common final answer across the runs.

SIMPLE EXPLANATION

Since a single reasoning path can go wrong, generating several independent reasoning paths and "voting" on the final answer catches errors that any one path might make.

💡 Like asking 5 different accountants to independently audit the same numbers and going with the answer most of them agree on — a single mistake is less likely to survive a majority vote.

EVERYDAY EXAMPLE

Run a tricky math word problem through the same CoT prompt 5 times at temperature 0.7; if 4 out of 5 runs land on "42," that's your answer, even if one run made an arithmetic slip.

WHY IT MATTERS

Self-consistency measurably improves reliability on hard reasoning tasks, at the cost of running the same prompt multiple times (more tokens, more latency, more cost).

COMMON MISTAKES

Using self-consistency at temperature 0, where every run produces an identical (not independent) answer, defeating the purpose.

BEST PRACTICES

Use a moderate temperature so runs genuinely vary; reserve this technique for high-stakes or unusually hard reasoning tasks given the extra cost.

SUMMARY

Self-consistency trades extra compute for higher reliability by "voting" across multiple independent reasoning attempts.

🔥 3.7 ReAct

DEFINITION

ReAct (Reasoning + Acting) is a prompting pattern where the model alternates between reasoning steps ("Thought") and actions ("Action," like calling a tool), observing results before continuing.

SIMPLE EXPLANATION

Instead of reasoning all at once and then acting, ReAct interleaves the two: think a bit, take an action (e.g., search, calculate), observe the result, think again, act again — much closer to how a person actually solves an open-ended problem.

Thought



Action



Observation



💡 Like troubleshooting a car problem: think about a possible cause, test it, observe the result, then think again based on what you just learned — not diagnosing the whole problem upfront in one guess.

EVERYDAY EXAMPLE

An agent answering "What's the weather where the 2024 Olympics were held?" might:

Thought: "I need to know the host city" → Action: search "2024 Olympics host city" → Observation: "Paris" → Thought: "Now I need Paris weather" → Action: check weather tool.

WHY IT MATTERS

ReAct is foundational to how most modern AI agents (Module 08) work — it's the core loop behind tool-using, multi-step assistants.

COMMON MISTAKES

Using ReAct for simple tasks that don't actually need external actions or multiple reasoning-action cycles — pure overhead for no benefit.

BEST PRACTICES

Use ReAct-style prompting when a task genuinely requires interleaving reasoning with real tool calls or lookups, not for single-shot answerable questions.

SUMMARY

ReAct interleaves reasoning and action in a loop — the backbone pattern behind most tool-using AI agents.

🌳 3.8 Tree of Thoughts (Concept)

DEFINITION

Tree of Thoughts (ToT) is a prompting/search strategy where the model explores multiple different reasoning branches at each step, evaluates them, and prunes weaker paths — like a search tree instead of a single linear chain.

SIMPLE EXPLANATION

Where chain-of-thought is one line of reasoning, tree of thoughts is several candidate lines explored in parallel, with the model (or a controlling process) periodically evaluating "is this branch promising?" before continuing down it.

Problem

```
|— Branch A (promising → continue)
|— Branch B (weak → pruned)
|— Branch C (promising → continue)
```

💡 Like a chess player considering several possible opening moves, briefly evaluating each, and only deeply analyzing the most promising ones — instead of committing to the first move that comes to mind.

EVERYDAY EXAMPLE

For a hard puzzle with multiple plausible approaches, ToT-style prompting might generate 3 different strategies, briefly evaluate each for feasibility, then fully develop only the most promising one.

WHY IT MATTERS

ToT-style exploration is especially useful for problems with many plausible-but-wrong paths, where a single linear chain-of-thought is prone to committing early to a dead end.

COMMON MISTAKES

Applying ToT to simple problems where a single reasoning chain is already reliable — the extra exploration is pure cost with no benefit.

BEST PRACTICES

Reserve tree-of-thoughts-style prompting for genuinely open-ended, high-uncertainty problems (planning, complex puzzles) rather than routine tasks.

SUMMARY

Tree of Thoughts explores and evaluates multiple reasoning branches rather than committing to one linear chain — powerful but expensive, best reserved for hard,

ambiguous problems.

🔗 3.9 Reflection

DEFINITION

Reflection prompting asks the model to review, critique, and revise its own previous output before finalizing an answer.

SIMPLE EXPLANATION

After generating an initial answer, you prompt the model again with something like "Review your answer above for errors and improve it" — giving it a second pass specifically focused on self-correction.

💡 *Like proofreading your own essay after a first draft — the act of rereading with a critical eye catches mistakes the first pass missed, even though it's the same writer both times.*

EVERYDAY EXAMPLE

Generate code, then prompt: "Review this code for bugs and edge cases you may have missed, then provide a corrected version."

WHY IT MATTERS

Reflection catches a meaningful share of errors that the first-pass answer misses, at the cost of an extra generation round.

COMMON MISTAKES

Asking for reflection without specific critique criteria ("just review it") — a more targeted prompt ("check for off-by-one errors and null handling") often works better.

BEST PRACTICES

Give the reflection step specific things to check for, relevant to common failure modes of that particular task.

SUMMARY

Reflection is a self-review pass that catches errors the first answer missed — a cheap way to meaningfully raise output quality.

🔗 3.10 Prompt Chaining

DEFINITION

Prompt chaining breaks a complex task into a sequence of smaller prompts, where the output of one becomes the input to the next.

SIMPLE EXPLANATION

Instead of one giant prompt trying to do everything at once, you split the task into stages (e.g., extract → summarize → format) each handled by its own focused prompt — often more reliable than one mega-prompt.

Prompt 1: Extract



Prompt 2: Summarize



Prompt 3: Format

💡 *Like an assembly line instead of one worker doing every step — each station focuses on one job well, and the output moves to the next station, instead of one overloaded worker trying to do everything at once.*

EVERYDAY EXAMPLE

A document-processing pipeline: Prompt 1 extracts key facts from a raw document, Prompt 2 summarizes those facts, Prompt 3 formats the summary into a specific report template.

WHY IT MATTERS

Chaining improves reliability on complex tasks (each stage is simpler and easier to verify), and makes debugging much easier — you can see exactly which stage produced a bad result.

COMMON MISTAKES

Over-chaining trivial tasks into unnecessary stages, adding latency and cost with no reliability benefit.

BEST PRACTICES

Chain when a task has genuinely distinct stages with different concerns; keep each link in the chain focused and independently testable.

SUMMARY

Prompt chaining splits a complex task into a sequence of focused prompts — more reliable and debuggable than one giant do-everything prompt.

4. Visual Learning

📊 Technique Comparison

Technique	Best for	Extra cost
Zero-shot	Simple, well-understood tasks	None
Few-shot	Consistent format/style, classification	Low-medium (examples)
Chain-of-Thought	Math, logic, multi-step reasoning	Medium (longer output)
Self-Consistency	High-stakes reasoning accuracy	High (multiple runs)
ReAct	Tool-using, multi-step agent tasks	Medium-high (loop iterations)
Tree of Thoughts	Open-ended, high-uncertainty problems	Very high (branch exploration)
Reflection	Catching errors in a first draft	Low-medium (one extra pass)
Prompt Chaining	Complex, multi-stage tasks	Medium (multiple calls)

🔫 Shot-based Techniques at a Glance

Shot-based Prompting

└─ Zero-shot (0 examples)

└─ One-shot (1 example)

└─ Few-shot (2-10 examples)

🌳 Chain vs. Tree Reasoning

Chain-of-Thought:

Step 1 → Step 2 → Step 3 → Answer (one path)

Tree of Thoughts:

Problem

└─ Branch A → Answer A

└─ Branch B → Answer B

└─ Branch C → Answer C

(explore several, keep the best)

5. Real World Applications

Product	How these techniques show up
Claude Code / Cursor / Copilot	ReAct-style loops (plan → run tool → observe → replan) drive autonomous coding tasks.
Customer support classification	Few-shot examples keep intent classification and routing consistent across thousands of tickets.
Math/reasoning tutoring apps	Chain-of-thought prompting shows work, which both improves accuracy and helps students follow the logic.
Research and planning agents	Tree-of-thoughts-style exploration considers multiple strategies before committing to one plan.
Content generation pipelines	Prompt chaining splits draft → fact-check → tone-adjust → format into separate, auditable stages.

6. Practical Examples

Beginner: Zero-shot vs. few-shot

Ask an LLM to classify 3 short product reviews as positive/negative/neutral with no examples, then again with 3 example classifications shown first. Compare consistency of the labeling style.

Beginner: Adding "let's think step by step"

Give the model a multi-step word problem directly, then again with "let's think step by step" appended. Compare accuracy and see the reasoning trace appear.

Intermediate: Reflection pass

Ask for a short piece of code, then follow up with "Review this for bugs and edge cases, then give a corrected version." Compare the two versions.

Advanced: Manual prompt chain

Take a messy paragraph of notes. Chain 3 prompts manually: (1) extract key facts as a list, (2) turn the list into a short summary, (3) format the summary as a formal memo. Compare against asking for all three in one single prompt.

7. Code Examples

🐍Python — Few-shot classification prompt

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

few_shot_prompt = """Classify sentiment as Positive, Negative, or Neutral.

Review: "Amazing quality, fast shipping!"
Sentiment: Positive

Review: "Broke after one day, terrible."
Sentiment: Negative

Review: "It's fine, does what it says."
Sentiment: Neutral

Review: "Cold food but the staff was great."
Sentiment: """

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=10,
    temperature=0,          # low temp: consistent classification
    messages=[{"role": "user", "content": few_shot_prompt}]
)

print(response.content[0].text)
```

📄JavaScript — Prompt chaining (extract → summarize)

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

async function ask(prompt) {
    const res = await client.messages.create({
        model: "claude-sonnet-4-6",
        max_tokens: 300,
        messages: [{ role: "user", content: prompt }],
    });
    return res.content[0].text;
}

// Stage 1: extract key facts
const facts = await ask(`Extract the 3 most important facts from this text:\n${rawText}`);

// Stage 2: summarize using Stage 1's output as input
const summary = await ask(`Summarize these facts in 2 sentences:\n${facts}`);

console.log(summary);
```

🐍Python — Self-consistency (simplified voting)

```
# pip install anthropic
import anthropic
from collections import Counter

client = anthropic.Anthropic(api_key="YOUR_API_KEY")
prompt = "A store had 23 apples, sold 8, then got 15 more. How many now? Let's think step by step, then give ONLY the final number."

answers = []
for _ in range(5):
    r = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=200, temperature=0.7,
        messages=[{"role": "user", "content": prompt}]
    )
    last_line = r.content[0].text.strip().split("\n")[-1]
    answers.append(last_line)
```

```
# Majority vote across 5 independent reasoning attempts  
print(Counter(answers).most_common(1))
```

**Tip**

Prompt chaining and self-consistency both mean multiple API calls per task — factor that into your cost and latency estimates (Module 01, Section 6).

8. Common Mistakes

- **Using few-shot examples that are too similar** to each other, giving no real coverage of edge cases.
- **Applying chain-of-thought to trivial tasks**, adding cost/latency with no accuracy benefit.
- **Running self-consistency at temperature 0**, where every run is identical and voting is meaningless.
- **Using ReAct-style loops for tasks with no real external actions needed** — pure unnecessary overhead.
- **Asking for reflection with no specific critique criteria**, getting a shallow "looks good" review.
- **Over-chaining simple tasks** into unnecessary multi-stage pipelines.

9. Best Practices

- **Start simple, escalate as needed:** zero-shot → few-shot → chain-of-thought → self-consistency, only adding complexity when accuracy demands it.
- **Pick diverse, representative few-shot examples**, not near-duplicates.
- **Reserve expensive techniques** (self-consistency, tree of thoughts) for genuinely hard, high-stakes problems.
- **Give reflection prompts specific criteria** to check for, not just "review this."
- **Chain prompts along genuinely distinct stages**, keeping each link focused and independently testable.

10. Hands-on Activities

Easy 5 exercises

1. Write a zero-shot classification prompt, then convert it to few-shot with 3 examples. Compare consistency across 3 test inputs.
2. Add "Let's think step by step" to a multi-step reasoning question and observe the reasoning trace.
3. Ask a model to generate a short paragraph, then apply a reflection prompt asking it to improve clarity.
4. Manually chain 2 prompts: summarize a paragraph, then translate the summary into a different tone (formal/casual).
5. Write out (on paper) a ReAct-style thought/action/observation trace for a simple real-world task, like planning a grocery run.

Intermediate 3 exercises

1. Run the same reasoning-heavy question 5 times at temperature 0.7, and manually tally which answer appears most often (self-consistency by hand).
2. Build a 3-stage prompt chain for a real task you have (extract → transform → format) and compare it against a single mega-prompt version.
3. Design 3 few-shot examples deliberately covering an easy case, a tricky edge case, and an ambiguous case for a classification task.

Challenge 2 exercises

1. Take a genuinely hard, multi-approach problem and sketch (on paper) a tree-of-thoughts exploration: 3 candidate approaches, a brief evaluation of each, and which one you'd pursue further and why.
2. Compare, on the same hard reasoning question, plain zero-shot vs. chain-of-thought vs. self-consistency (5 runs). Write up which technique performed best and why you think that happened.

11. Mini Project

Goal

Build a small "technique comparison" script that runs the same hard question through zero-shot, chain-of-thought, and self-consistency, and prints all results side by side.

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK
- One multi-step reasoning question of your choice (math or logic works well)

Step-by-Step Instructions

1. Write a function that calls the API with a given prompt and temperature.
2. Run your question zero-shot (temperature 0) and print the answer.
3. Run it again with "let's think step by step" appended (temperature 0) and print the answer.
4. Run it 5 times with the step-by-step version at temperature 0.7, and tally the most common final answer (self-consistency).
5. Print all three results together, labeled clearly.

Expected Output

```
=== Zero-shot ===
[answer]

=== Chain-of-Thought ===
[reasoning + answer]

=== Self-Consistency (5 runs) ===
Run 1: [answer]  Run 2: [answer]  Run 3: [answer]
Run 4: [answer]  Run 5: [answer]
Majority answer: [answer]
```

Possible Improvements

- Add a reflection pass on top of the chain-of-thought answer and compare again.
- Track and print the token cost of each technique to see the real cost/accuracy tradeoff.
- Test with a genuinely hard question where zero-shot is known to fail, to make the comparison more dramatic.

12. Review Questions

Multiple Choice (10)

1. Zero-shot prompting means:

- A) Using zero temperature B) Giving the model no examples, only an instruction C) Using zero tokens D) Skipping the system prompt

2. Few-shot prompting is most useful for:

- A) Teaching a pattern via multiple examples B) Reducing token cost to zero C) Skipping instructions entirely D) Increasing temperature automatically

3. Chain-of-thought prompting primarily helps with:

- A) Reducing max_tokens B) Multi-step reasoning and math accuracy C) Lowering API cost D) Choosing a persona

4. Self-consistency requires:

- A) Temperature 0 always B) Running the same prompt multiple times and voting on the answer C) A single deterministic run D) No reasoning steps at all

5. ReAct stands for reasoning combined with:

- A) Reflection B) Acting (taking actions/tool calls) C) Reviewing D) Ranking

6. Tree of Thoughts differs from Chain-of-Thought by:

- A) Using no reasoning at all B) Exploring multiple reasoning branches instead of one linear path C) Requiring zero-shot only D) Being cheaper to run

7. Reflection prompting asks the model to:

- A) Ignore its previous answer B) Review and revise its own prior output C) Lower its temperature D) Skip the task entirely

8. Prompt chaining is best suited for:

- A) Single trivial questions B) Complex, multi-stage tasks C) Reducing the number of API calls D) Avoiding context windows

9. Which technique has the HIGHEST extra cost typically?

- A) Zero-shot B) One-shot C) Tree of Thoughts D) Simple instruction

10. Self-consistency at temperature 0 is problematic because:

- A) It costs nothing B) Every run produces an identical result, defeating the purpose of voting C) It's too fast D) It requires no reasoning

Identification (5)

1. ____ provides multiple examples of a task before asking the model to perform it on new input.

2. ____ elicits visible intermediate reasoning before a final answer.

3. ____ alternates between reasoning steps and taking actions, observing results along the way.

4. ____ explores multiple reasoning branches and prunes weaker ones.

5. ____ breaks a complex task into a sequence of smaller, connected prompts.

True or False (5)

1. Zero-shot prompting always produces worse results than few-shot prompting.

2. Chain-of-thought prompting can improve accuracy on multi-step math problems.

3. Self-consistency works best when run at temperature 0.

4. ReAct is a core pattern behind many modern tool-using AI agents.

5. Reflection prompting means asking the model to review and improve its own previous answer.

Situational (5)

1. Your classification prompt gives inconsistent label formats across similar inputs. Which technique would you try first?

2. Your model consistently makes arithmetic errors on multi-step word

problems. What technique would you add to the prompt?

3. You need very high confidence on a single, high-stakes reasoning answer and can afford extra API calls. What technique fits?

4. Your agent needs to search the web, then reason about the results, then search again based on what it found. What pattern describes this?

5. Your single "do everything" prompt keeps producing inconsistent results on a complex multi-stage task. What would you try restructuring it into?

Answer Key

Multiple Choice: 1-B, 2-A, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-C, 10-B

Identification: 1-Few-shot prompting, 2-Chain-of-thought, 3-ReAct, 4-Tree of Thoughts, 5-Prompt chaining

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Few-shot prompting — showing 2-3 consistent examples usually fixes formatting drift.
2. Chain-of-thought ("let's think step by step") to surface and reduce reasoning errors.
3. Self-consistency — run multiple reasoning attempts and take the majority answer.
4. ReAct — the interleaved reasoning/action/observation loop.
5. Prompt chaining — split into focused, sequential stages instead of one mega-prompt.

13. Cheat Sheet

Shot-based

Zero-shot — instruction only.

One-shot — 1 example.

Few-shot — 2-10 diverse examples.

Reasoning Boosters

Step-by-step / CoT — visible reasoning trace.

Self-consistency — vote across multiple runs.

Tree of Thoughts — explore multiple branches.

Agentic Patterns

ReAct — Thought → Action → Observation loop, foundational for agents (Module 08).

Quality & Structure

Reflection — self-review and revise.

Prompt chaining — split into focused, sequential stages.

14. Key Takeaways

- Start with zero-shot for simple tasks; escalate to few-shot, then reasoning techniques as accuracy demands increase.
- Few-shot examples should be diverse and representative, not near-duplicates.
- Chain-of-thought elicits visible reasoning that measurably improves multi-step accuracy.
- Self-consistency trades extra cost for reliability by voting across multiple independent reasoning runs — requires non-zero temperature to work.
- ReAct interleaves reasoning and action, forming the backbone of most tool-using AI agents.
- Tree of Thoughts explores multiple reasoning branches for hard, open-ended problems — powerful but expensive.
- Reflection is a cheap self-review pass that catches errors the first answer missed.
- Prompt chaining splits complex tasks into focused, sequential, individually testable stages.

15. Additional Resources

Official Documentation

- Anthropic — Prompt engineering: chain-of-thought guidance (docs.claude.com)
- Anthropic — Multishot prompting guidance (docs.claude.com)

Research Papers

- "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (Wei et al., 2022)
- "Self-Consistency Improves Chain of Thought Reasoning in Language Models" (Wang et al., 2022)
- "ReAct: Synergizing Reasoning and Acting in Language Models" (Yao et al., 2022)
- "Tree of Thoughts: Deliberate Problem Solving with Large Language Models" (Yao et al., 2023)

Tools

- Anthropic Console — for quickly A/B testing prompting techniques side by side.

Communities & Further Learning

- Provider prompt libraries with worked examples of few-shot and chain-of-thought prompts across use cases.